```
@RestController
public class QuizController {
     @Autowired
     private QuestionRepository repo;

     // mapping methods

}
```

The *add()* method maps *POST /question* API, accepts Question as RequestBody of the *HTTP Post* request, and saves it in the repository and thereby in the database.

```
     @PostMapping("/question")
     public ResponseEntity<Question> add(@RequestBody
Question question) {
       question = repo.save(question);
       return ResponseEntity.status(HttpStatus.CREATED).
body(question);
     }
```

The *generate()* method maps *GET /questions* API, extracts subject and topic parameters from the request URI, and fetches the matching question IDs from the repository. It randomly picks ten question IDs and returns the corresponding questions. Randomisation can also be done in the database itself, but not all databases support such an operation. To make the implementation work for any RDBMS, we chose to randomise the question IDs in the service layer.

```
     @GetMapping("/questions")
     public ResponseEntity<List<Question>> generate(@
RequestParam("subject") String subject,
          @RequestParam("topic") String topic) {
       List<Integer> ids = repo.findQidBySubjectAndTopic(subje
ct, topic).stream().map(qn -> qn.getQid())
              .collect(Collectors.toList());
       Collections.shuffle(ids);
       List<Question> questions = repo.findByQidIn(ids.
subList(0, 10));
       return ResponseEntity.status(HttpStatus.OK).
body(questions);
     }
```

The *evaluate()* method maps *POST /answers* API and gets the list of answers from the request body. For each of the answers, the method checks with the repository if it is right or wrong, and returns the final computed score.

```
     @PostMapping("/answers")
     public ResponseEntity<Score> evaluate(@RequestBody
List<Answer> answers) {
       int rights = 0;
       for (Answer answer : answers)
          rights += (int) repo.countByQidAndAnswer(answer.
getQid(), answer.getOption());
       Score score = new Score(answers.size(), rights);
       return ResponseEntity.status(HttpStatus.CREATED).
body(score);
     }

}
```

The Answer and Score objects are simple POJOs.

```
public class Answer {
     private int qid;
     private int option;

     ...
}

public class Score {
     private int total;
     private int right;

     ...
}
```

Step 5: Configuration

The *application.properties* provides the configuration for the data source. Since we are using H2 in this project, the following configuration works for it:

```
spring.datasource.url=jdbc:h2:mem:quiz-db
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.jpa.properties.hibernate.dialect=org.hibernate.
dialect.H2Dialect
spring.jpa.hibernate.ddl-auto= update
spring.h2.console.enabled=true
```

The Tomcat server runs on the default 8080 port. We added configuration to add */quiz* as the URL prefix.

```
server.port=8080
server.servlet.contextPath=/quiz
```

The complete code of the application is available at *https://bitbucket.org/glarimy/glarimy-university/src/master/glarimy-boot/*.

Step 6: Build, run, test, and deploy

Build the application and access the database at *http://*